

Beyond Naive RAG: Implementing Probabilistic Corrective Frameworks for Financial and Business Analysis

Dishen Yang (dy2525), Siwen Chen (sc5552), and Jiayi He (jh5111)

1. Introduction & Problem Statement

1.1 Context & Background

In high-stakes financial environments such as M&A feasibility analysis, post-investment risk management, and corporate valuation, the professional analysts rely heavily on extracting precise metrics from massive, unstructured document repositories. Generative AI and Large Language Models offer immense potential to automate this data extraction and synthesize cross-year performance comparisons.

1.2 Problem Statement

Despite the widespread adoption of Baseline RAG to ground LLMs in external knowledge, traditional RAG architectures face critical limitations in precision-critical domains. Standard RAG systems rely heavily on the retrieved documents. When the retrieved documents are wrong, the produced answers would be inaccurate due to the lack of self-monitoring mechanisms. These standard RAG systems show a lack of flexibility as they treat all retrieved documents as equally valid. In the financial sector, if a semantic search mistakenly retrieves wrong information, a standard RAG system will confidently hallucinate an answer based on this erroneous data. This risk is unacceptable for analysts in high stake environments. In this case, a Corrective RAG (CRAG) would act as a safeguard layer and evaluate the document before it is sent to the model for answer generation. Thus, we would like to explore in this project how would the use of CRAG reduce the hallucinations and improve the efficiency of answer generation of LLMs under high-stake environments.

1.3 Scope Definition

To ensure project feasibility, the scope of this project is strictly defined as developing a “Probabilistic CRAG” pipeline. This involves building a custom probabilistic retrieval evaluator using statistical classification methods to assign a continuous confidence score $P(Relevant|q, d)$ in $[0, 1]$ to retrieve financial chunks. We will implement a soft-routing mechanism with defined thresholds to either refine internal knowledge or trigger a web-search fallback API when internal data is outdated (Other methods may be utilized in the process of research).

However, we will not train a foundational LLM from scratch, nor will we build a full-scale commercial UI or process real-time, high-frequency algorithmic trading data. The focus of the project remains on the design of the algorithmic pipeline and the comparative evaluation against baseline RAG.

1.4 Research Question

To what extent does integrating a continuous probabilistic retrieval evaluator and table-aware document processing into a Corrective RAG architecture reduce hallucination rates and improve numerical reasoning accuracy on complex financial documents, compared to a standard RAG baseline?

In the same time, we would like to also explore how much latency would change by using CRAG compared with a standard RAG baseline.

2. Literature Review & AI Trends

2.1 Current AI Trends

The generative AI landscape is currently shifting from static RAG architectures to dynamic, agentic workflows. A prominent trend in enterprise AI is the implementation of self-correcting mechanisms and confidence thresholds to solve the problem of hallucinations. Industry leaders are actively utilizing these intelligent routing systems. For instance, IBM Watsonx integrates Granite models with external search APIs to evaluate document quality dynamically, rewriting search queries when internal knowledge on sensitive topics is insufficient. Similarly, architectures like CRAG are being adopted across disciplines, from agricultural genomics (CRAG-JIC-MPIPZ alliance) to financial risk advisory, where cross-validation is used to resolve contradictory client financial data.

2.2 Literature Review & Limitations of Existing Approaches

Traditional RAG assumes that retrieved context is always accurate. As noted in recent literature, this leads to catastrophic failures in high-stake fields when the retrieval corpus contains mixed-quality data.

To address this issue, the CRAG framework introduced a retrieval evaluator to assess document relevance. Empirical benchmarks by Yan et al. (2024) demonstrate that CRAG improves factual accuracy by 8% to 36.6% over standard RAG, with the most significant gains (up to 36.6% on PubHealth) observed in mixed-quality corpora, which is a scenario that mirrors enterprise financial databases. Commercial implementations corroborate these findings: CustomGPT.ai reported a 34% reduction in enterprise hallucinations using confidence thresholds, and Anpu Labs reduced customer service escalations by 22% through automated fact-checking. Furthermore, research on BlackRock's corporate knowledge retrieval highlights CRAG's superiority in complex financial queries, utilizing web-search fallback when internal SEC filings are outdated.

While existing CRAG models do improve the robustness of retrieval, their standard retrieval evaluators typically rely on a binary classification or prompt-based LLM judgments. However, this binary approach may not be enough under complex financial settings. Our project addresses this limitation by introducing a probabilistic continuous function to the CRAG evaluator. By outputting a probability score rather than a binary tag, our architecture allows for more tunable, risk-adjusted thresholding (three potential categories: correct, ambiguous, incorrect) that aligns with the requirements of financial and business analysis.

3. Proposed Methodology

3.1 Data Source

The document data planned to be used in this project mainly comes from the EDGAR public database of the U.S. Securities and Exchange Commission (SEC). EDGAR includes a number of financial documents submitted by listed companies in accordance with the law, including 10-K annual report, 10-Q quarterly report, 8-K interim report and DEF 14A agent statement, etc., and relevant documents can be publicly obtained. This kind of document is usually long, complex in format, and contains a mixed presentation of text content and table information, so it is suitable for evaluating the performance of the Q&A system in real financial scenarios.

3.2 Baseline Model

This project will first build a standard RAG pipeline as a baseline system. The baseline model is aimed at selected financial documents and adopts a more common and low-cost text-based RAG architecture,

which mainly includes three stages: document processing, vector retrieval and answer generation.

- In the document processing stage, the system first extracts and preprocesses the text of the selected financial file, and then blocks (fixed-size chunking) according to a fixed length to construct retrievable text units.
- In the retrieval stage, the system uses the pre-trained text embedding model to vectorize the text block and establish a vector index. When the user enters the question, the system converts the question into a vector representation and returns the most relevant text blocks through similarity retrieval as the basis for the generation of answers.
- In the answer generation stage, the system will input the retrieved text fragment into the large language model with the user's question, and the model will generate the final answer based on the given context.

3.3 Proposed Corrective RAG Architecture

Standard RAG usually enters the answer generation stage directly after the retrieval is completed, and there is a lack of judgment on whether the retrieval results are sufficient to support the answer. In response to this problem, it is planned to add CRAG as an intermediate evaluation step between retrieval and generation to judge the relevance and adequacy of the retrieval results. We plan to utilize a method called probabilistic thresholding that assigns a confidence score between 0 and 1 to the retrieved context rather than assigning a binary class. If the score falls below a certain threshold, a fallback mechanism may be triggered. These fallback mechanisms could be web-searching or further expanding the retrieval scope. The thresholding would ensure that the system refuses to generate unreliable answers.

3.4 Other Proposed Improvements

The baseline plan can complete the basic process of financial document questions and answers, but it still has certain limitations. For example, fixed-length blocks may cut off the semantic integrity of the content and affect the retrieval effect; the lack of specialized processing of table information may reduce the system's ability to answer numerical questions; the standard RAG pipeline usually lacks the judgment of the quality of the retrieval results, and when the retrieval evidence is insufficient or inaccurate, it may still generate unreliable answers. Therefore, it is planned to improve the RAG pipeline from the following directions.

(1) Proposed Improvement Module I: Table-aware document processing.

A large amount of key information in financial documents is presented in the form of tables, such as balance sheets, profit statements and key financial indicators. The baseline scheme directly converts the table into ordinary text, which is easy to lose the correspondence of the row. To this end, we plan to introduce a table detection and parsing module to convert the identified tables into clearer structured representations, such as Markdown or JSON, and incorporate them into the subsequent retrieval process. The purpose of this improvement is to improve the system's ability to retain table information and improve the quality of answering questions involving financial indicators and numerical content.

(2) Proposed Improvement Module II: Semantic block strategy.

The baseline scheme adopts fixed-length blocks, which may interrupt semantically complete paragraphs and affect the quality of the retrieval results. To this end, it is planned to explore the blocking method based on semantic information to help determine the block boundary through the semantic correlation between sentence-level representation and adjacent content, so as to make each text block more complete and coherent in content. The improvement aims to optimize the organization of the knowledge base and

provide more suitable text units for subsequent retrieval.

3.4 Innovation Points

Unlike the general RAG systems, this project focuses on financial document scenarios, and the data processed includes both the body content and a large amount of table information. Standard RAG usually generates answers directly after retrieval, and lacks judgment on whether the retrieval results are sufficient and reliable. This project plans to add CRAG as evaluation steps between retrieval and generation, and trigger re-retrieval or supplementary retrieval when there is insufficient evidence, so as to reduce the risk of the system generating unreliable answers when there is insufficient information. Not only will we be incorporating a Corrective RAG into the system, we will also be working on other methods to improve retrieval accuracy and latency. The system design will focus on the characteristics of information dispersion, numerical density and complex structure in financial documents, so it is closer to the real application needs. Our proposed improvements further considers the structured analysis and retrieval of the table content, so that the system can not only use text evidence, but also better process numerical information such as financial indicators and statement data. This project does not rely on a single complex model, but gradually adds more modules that build on the current CRAG architecture.

The methodology outlined above represents our primary architecture. During the development phase, we may also explore other potential integrations that build upon the current CRAG architecture, such as dynamic routing or self-reflection tokens that can further optimize the fallback mechanisms.

4. Implementation & Real-World Value (The "Impact")

4.1 Real-World Applications

This system is designed directly to be utilized for quantitative analysts, business analysts, financial or corporate strategy analysts who need to extract precise information from dense business and financial documents. The financial field especially contains a large number of annual reports, quarterly reports, financial disclosure documents and investor materials. These documents are long and information is scattered. Many key contents not only appear in the text, but also in tables or notes. For users, the real difficulty is often not to obtain the document itself, but how to quickly locate effective information and get accurate answers based on the content of the document. Standard RAG models are easy to get temporal hallucinations when facing these documents, which may lead to catastrophic results for business decisions backed by the wrong data. The Corrective RAG architecture, on the other hand, would act as a safeguard against such mistakes and actively calculate a confidence probability to prevent the model from outputting information from the wrong documents. Based on this background, this project intends to build a chat system for financial document questions and answers, so that users can directly query the key information in the report through natural language, such as the company's business performance, changes in financial indicators and risk disclosure. The focus of the project is not only to generate answers, but also to make the answers based on specific document evidence as much as possible, so as to improve the accuracy and credibility of the system in financial scenarios.

The system can be used in financial research, report reading assistance and information preliminary screening and other scenarios. For example, users can ask questions around a financial report, quickly obtain an explanation of an indicator, a summary of a certain disclosure, or locate the document part related to the problem. The value of the system is mainly reflected in improving the efficiency of information retrieval, reducing the cost of reading, and providing clearer references for subsequent

analysis.

4.2 Live Demo Expectations & Ideas:

Our primary plan for the demo is to build an interactive web interface using existing tools such as Streamlit to showcase a side-by-side comparison. We will be building a chatbot and using tools such as LangChain or LlamaIndex to connect the built chatbot with the interphase and the database. For the LLM, we will choose an API from current state-of-the-art artificial intelligence companies like OpenAI, Anthropic, or Google. We will also be building a local small-scale vector database for demonstration purposes.

4.3 Scalability & Resource Efficiency:

By filtering out unnecessary and unrelated documents during the retrieval process before they are sent to the LLM for answer generation, our architecture reduces API token expenditure and hence increasing the efficiency, reducing costs for enterprise-level scaling.

5. Evaluation & Metrics

5.1 Quantitative Metrics

We plan to use the following as our evaluation frameworks:

- Retrieval accuracy and faithfulness score: this score measures how tightly the final answer follows the given financial or business context in the prompt given
- Routing precisions: this is a metric that we could use to measure how many times that the CRAG architecture correctly identified an irrelevant or wrong document and the times it triggered the false alarm.
- Latency: track the response time by comparing the standard RAG with the new CRAG architecture to see the change in efficiency.

The specific evaluation dimensions include:

- Baseline comparison: compare the accuracy of the baseline system and the improvement system on the same group of questions.
- Local ablation experiment: ablation and comparison of key improvement modules, such as comparing the performance differences before and after whether to add the table analysis module and whether to add the retrieval quality evaluation mechanism, so as to analyze the actual role of the main modules.
- Analysis by question type: count the accuracy rate of answers to text, table and comprehensive questions respectively, observe the performance differences of the system under different question types, and analyze the follow-up optimization direction accordingly.

5. 2 User Experience Improvements

We think this architecture would improve the users' experience by improving their trust in the model. By providing the confidence score alongside with the answer, the analysts would instantly get the reliability of the data provided by the model, avoiding possible hallucinations and increasing the practicality of AI models under high-stakes environments.

6. Project Plan & Risk Management

6.1 Realistic Timeline & Milestones

(1) Week 1

- Download and sort out documents from the SEC EDGAR database; build a development environment; complete the construction and preliminary testing of the baseline RAG pipeline.
- Milestone: Successfully run the baseline system.

(2) Week 2: Core Improvement Stage

- Develop the table detection and structured analysis module, and access the retrieval process; start on working the initial CRAG architecture; start to build evaluation questions and answers (text class, table class, comprehensive class); conduct preliminary tests on table-related problems.
- Milestone: The table perception module can be initially run, and the system can handle some problems containing table information.

(3) Weeks 3–4: Core Improvement Stage and Evaluation Stage

- Evaluate the retrieval quality and regression mechanism, and further explore the semantic block strategy; integrate the proposed improved modules into the complete pipeline; run the baseline comparison experiment. The number of improved modules should be implemented according to time concerns.
- Milestone: Complete the improvement methods and the complete system can be run, and the main evaluation data collection has been completed.

(4) Week 5: Presentation Preparation Stage

- Build a Chatbot interactive interface; sort out the experimental results; design sample questions for display.
- Milestone: The system has a mobile interactive interface and basic display content.

(5) Week 6: The final stage.

- Write reports, make presentation slides, and organize GitHub repositories.
- Milestone: All deliverables of the project are completed according to the rubric.

6.2 Required Resources & Availability

- Computing resources: Use the GPU resources provided by Google Colab for development or other cloud compute resources.
- API service: We will utilize API from commercial platforms such as OpenAI, Anthropic, or Google.
- Data resources: We will be using sources such as SEC EDGAR financial documents that are publicly available resources with no permissions required or fees.
- Development tools (to be determined, below are examples of tools that we will use):
 - Vector search: FAISS.
 - Text embedding: SentenceTransformers (such as BGE series models).
 - Table detection and analysis: such as pdfplumber, Camelot, Docling, Microsoft Table Transformer, decided according to the actual situation.
 - Interactive interface: Streamlit or Gradio for system presentation and basic interaction.
 - Code management: GitHub.

6.3 Challenges

- (1) The accuracy of table analysis is unstable.

There are various table formats in financial documents. Some tables have complex structures and more nested levels. There may be differences in the analysis quality of open source tools, thus affecting the subsequent retrieval and answer effect.

- (2) The effect of the retrieval evaluation mechanism is difficult to guarantee.

CRAG-style retrieval quality evaluation depends on the model's ability to judge correlation. If the evaluation results themselves are not accurate enough, the return mechanism may be triggered by mistake or fail to trigger in time when the evidence is insufficient, thus affecting the overall stability of the system.

- (3) There is uncertainty about the performance of the open source model under the financial scenario.

The ability of the open source large language model to understand financial terms, numerical information and complex document content may be limited, and the effect in numerical reasoning and comprehensive answers may not be stable.

6.4 Coping strategy

- (1) For the table analysis problem, if the automatic analysis effect is not ideal, a semi-automatic method can be used to manually proofread a small number of key tables or manually mark the structure to ensure that the table problems in the evaluation data have usable structured content. At the same time, you can compare a variety of analysis tools and choose a scheme with better effect.

- (2) For the problem of retrieval evaluation mechanism, if the model-based evaluation method is too complex or the effect is unstable, it can be simplified to a rule judgment based on the similarity threshold. For example, when the maximum similarity of the retrieval results is lower than the set threshold, it automatically triggers re-retrieval or supplementary retrieval. This method is relatively simple to implement and the behavior is easier to control.

- (3) In response to the performance of the open source model, the project will retain the option of switching to commercial API. On the system architecture, large language model calls can be encapsulated into independent modules in order to switch between different models without affecting the implementation of other parts.

Minimum deliverable results: In the most conservative case, this project delivers at least one operable baseline RAG Q&A system and completes the implementation and access of at least one key improvement module. Get the comparative experimental results of the baseline system and the improved system, as well as a Chatbot interactive interface.

7. References

1. Chen, Z., Chen, W., Smiley, C., Shah, S., Borova, I., Langdon, D., ... & Wang, W. Y. (2021, November). *FinQA: A dataset of numerical reasoning over financial data*. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (pp. 3697-3711). <https://doi.org/10.48550/arXiv.2109.00122>.
2. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. Advances in neural information processing systems, 33, 9459-9474. <https://doi.org/10.48550/arXiv.2005.11401>

3. Sahin, S. (2024, May 19). *CRAG (Corrective Retrieval-Augmented Generation) in LLM: What it is & How it works*. Medium.
<https://medium.com/@sahin.samia/crag-corrective-retrieval-augmented-generation-in-llm-what-it-is-and-how-it-works-ce24db3343a7>
4. Smock, B., Pesala, R., & Abraham, R. (2022). *PubTables-1M: Towards comprehensive table extraction from unstructured documents*. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 4634-4642).
<https://doi.org/10.48550/arXiv.2110.00061>
5. Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). *Corrective retrieval augmented generation*. arXiv. <https://arxiv.org/abs/2401.15884>
6. Zhu, F., Lei, W., Huang, Y., Wang, C., Zhang, S., Lv, J., ... & Chua, T. S. (2021, August). TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In Proceedings of the 59th annual meeting of the Association for Computational Linguistics and the 11th international joint conference on natural language processing (volume 1: long papers) (pp. 3277-3287). <https://doi.org/10.48550/arXiv.2105.07624>

We used ChatGPT 5.4 and Gemini 3.1 Pro for idea refinement, information gathering, and structuring of the proposal.